

Name: Christine M. Khayat

Part 1: Connection Time Experiment

Paste your screenshot of the connection benchmark output below:

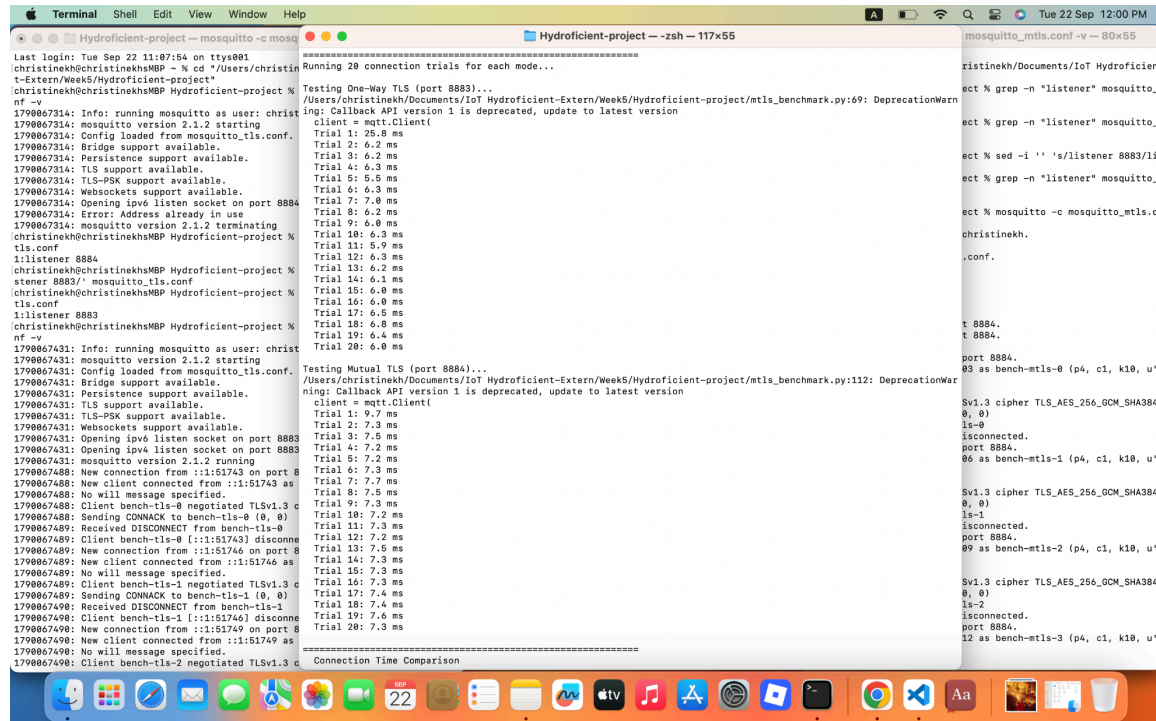
```
=====
Connection Time Comparison
=====

Trials: 20

One-Way TLS:
Average: 7.2 ms
Min: 5.5 ms | Max: 25.8 ms

Mutual TLS:
Average: 7.5 ms
Min: 7.2 ms | Max: 9.7 ms

Overhead: +0.3 ms (+3.8%)
=====
christinekh@christinekhsMBP Hydroficient-project %
```



Connection Time Results

Metric	One-Way TLS	Mutual TLS
Average connection time	7.2 ms	7.5 ms
Min connection time	5.5 ms	7.2 ms
Max connection time	25.8 ms	9.7 ms
Overhead	--	+0.3 ms (+3.8%)

Part 2: Message Latency Experiment

Paste your screenshot of the latency benchmark output below:

```
=====
christinekh@christinekhsMBP Hydroficient-project % python3 mtls_benchmark.py --mode latency
=====
mTLS Benchmark: Message Latency
=====
Sending 50 messages for each mode...

Testing One-Way TLS (port 8883)...
/Users/christinekh/Documents/IoT Hydroficient-Extern/Week5/Hydroficient-project/mtls_benchmark.py:238: DeprecationWar
ning: Callback API version 1 is deprecated, update to latest version
  sub_client = mqtt.Client(client_id=f"bench-sub-{port}", **MQTT_CLIENT_ARGS)
/Users/christinekh/Documents/IoT Hydroficient-Extern/Week5/Hydroficient-project/mtls_benchmark.py:261: DeprecationWar
ning: Callback API version 1 is deprecated, update to latest version
  pub_client = mqtt.Client(client_id=f"bench-pub-{port}", **MQTT_CLIENT_ARGS)
Connected. Publishing 50 messages...
Average latency: 1.0 ms

Testing Mutual TLS (port 8884)...
Connected. Publishing 50 messages...
Average latency: 1.0 ms

=====
Message Latency Comparison
=====
Messages: 50

One-Way TLS:
Average: 1.0 ms
Min: 0.5 ms | Max: 2.1 ms

Mutual TLS:
Average: 1.0 ms
Min: 0.5 ms | Max: 1.3 ms

Overhead: +0.0 ms (+1.7%)
=====
christinekh@christinekhsMBP Hydroficient-project %
```

Message Latency Results

Metric	One-Way TLS	Mutual TLS
Average latency	1.0 ms	1.0 ms
Min latency	0.5 ms	0.5 ms
Max latency	2.1 ms	1.3 ms
Overhead	--	+0.0 ms (+1.7%)

Part 3: Analysis

1. Connection Overhead

How many extra milliseconds does mTLS add to connection setup?

Your answer: mTLS adds **0.3 ms** to the average connection setup time.

The Grand Marina's devices connect once and stay connected for hours. Does this one-time cost matter?

Your answer: No. Since the Grand Marina's devices connect once and remain connected for hours, the **0.3 ms one-time overhead is very small and has minimal practical impact** on the system.

2. Message Latency

How many extra milliseconds does mTLS add per message?

Your answer: mTLS adds approximately **0.0 ms** of displayed average latency per message. The benchmark reports **+1.7% overhead**, but both average latency values round to **1.0 ms**.

Sensor readings are sent every 5 seconds (5,000 ms). What percentage of that interval is the mTLS overhead?

Your answer: Using the displayed **0.0 ms** overhead:

$$0.0 \div 5,000 \times 100 = \text{approximately } 0\%$$

Therefore, the mTLS overhead represents a **negligible portion of the 5-second sensor reporting interval**.

Recommendation to Marcus:

Based on the benchmark results, I recommend keeping mTLS enabled for the Grand Marina water monitoring system. mTLS increased average connection time by only 0.3 ms (3.8%), while message latency remained essentially unchanged. Because the HYDROLOGIC devices stay connected for hours, the small additional connection cost occurs only once and has minimal practical impact. The measured message latency overhead is also negligible compared with the 5-second sensor reporting interval. Therefore, mTLS provides stronger device authentication without significantly affecting the performance of the monitoring system.